



Reporte técnico de la aplicación móvil

Una guía para entender, mantener y extender la app

Audiencia: desarrolladores web con experiencia en JavaScript, PHP y/o Go.
No se asume conocimiento previo de Flutter ni de Dart.

Contenido

1.	Introducción y mapa mental	3
1.1.	¿Qué es Flutter y por qué importa entenderlo?	3
1.2.	¿Qué es Dart?	3
1.3.	Vistazo del repositorio	3
2.	Arquitectura: Clean Architecture versión Flutter	4
2.1.	Las tres capas	4
2.2.	Ejemplo: la feature auth	4
2.3.	Una sutileza: entidad vs modelo	5
3.	Stack de librerías: qué usamos y por qué	5
4.	La magia: generación de código en build time	7
4.1.	El comando sagrado	7
4.2.	@riverpod — providers tipados sin boilerplate	7
4.3.	@freezed — data classes inmutables con superpoderes	8
4.4.	@JsonSerializable — fromJson y toJson automáticos	8
4.5.	@DriftDatabase — la base de datos local	9
4.6.	Resumen de archivos .g.dart	9
5.	HTTP: cómo se habla con el backend	9
5.1.	El singleton DioClient	9
5.2.	El AuthInterceptor — el middleware que mete el token	10
5.3.	El backend esperado	11
6.	Estado global: Riverpod 3.x en 5 minutos	12
6.1.	El modelo mental	12
6.2.	¿Quién le pasa ref a quién?	12
6.3.	Ejemplo: el grid de áreas	12
7.	La pieza estrella: el sistema de grids reutilizable	13
7.1.	Lo que el grid te da gratis	13
7.2.	Anatomía del grid: cinco archivos por catálogo	14
7.3.	LaravelResourceController — el builder de URLs	14
7.4.	GridAction<T> — el menú de 3 puntos	15
7.5.	HasId y PaginatedResult<T> — los contratos mínimos	15
7.6.	El form genérico	16
8.	Almacenamiento local: Drift (SQLite tipado)	17
8.1.	El schema	17
8.2.	Provider y ciclo de vida	18
8.3.	Los datasources locales	18
9.	Notificaciones: recordatorios de exámenes	18
9.1.	Flujo general	18
9.2.	Zona horaria	19
10.	Router: navegación declarativa	19
11.	Recorrido guiado de los flujos clave	20
11.1.	Flujo 1: login con boleta y contraseña	20
11.2.	Flujo 2: listar áreas (admin)	20
11.3.	Flujo 3: editar un área	21
11.4.	Flujo 4: alumno abre la app sin internet	21
12.	Estructura detallada de carpetas	22
13.	Convenciones del proyecto (leer antes de tocar nada)	23
14.	Glosario rápido (Dart/Flutter → JS/PHP/Go)	24
15.	Resumen ejecutivo (para responder preguntas rápidas)	25

1. Introducción y mapa mental

Este reporte describe cómo está construida la app móvil **bitets** desde cero, asumiendo que el lector domina el desarrollo web con JavaScript, PHP o Go, pero nunca ha tocado Flutter o Dart. La idea es que después de leerlo puedas abrir cualquier archivo de `lib/`, entender qué hace, y explicar la app como si la hubieras escrito tú.

Cómo leer este documento. El documento está escrito como un tour: empezamos por el «qué es Flutter y Dart», seguimos por la arquitectura general, y luego bajamos a cada pieza con analogías concretas a JavaScript, Laravel/PHP y Go. La sección final («Recorrido guiado de los flujos clave») ata todo: si solo puedes leer una sección, lee esa.

1.1. ¿Qué es Flutter y por qué importa entenderlo?

Flutter es a las apps móviles lo que Next.js es a React: un framework que incluye un **runtime** propio (llamado Skia), un compilador y un set de componentes. Cuando haces `flutter run` se compila a binarios nativos de Android, iOS, Linux, macOS, Windows y Web. Aquí lo usamos para Android, iOS y escritorio, principalmente móvil.

A diferencia de React Native (que renderiza con componentes del sistema operativo), Flutter **dibuja** cada píxel: no hay View de Android ni UIView de iOS, todo es un widget de Flutter. Esto es importante porque muchos conceptos que en web son «el navegador los maneja» (layout, eventos, animaciones) aquí los maneja el runtime de Flutter.

1.2. ¿Qué es Dart?

Dart es a Flutter lo que TypeScript es a React: el lenguaje en el que escribes la app. La analogía útil:

Concepto web	Concepto Dart
TypeScript: tipado gradual	Dart: tipado fuerte, null safety
<code>async/await</code> con <code>Promise<T></code>	<code>async/await</code> con <code>Future<T></code>
<code>Array<T></code> , <code>Map<K,V></code>	<code>List<T></code> , <code>Map<K,V></code>
<code>class Foo extends Bar</code>	<code>class Foo extends Bar</code> (idéntico)
<code>interface Foo { ... }</code>	<code>abstract class Foo { ... }</code> o <code>mixin</code>
<code>const x = 1</code>	<code>final x = 1</code> o <code>const x = 1</code>
<code>null</code>	<code>null</code> (con null safety)
JSX/TSX	árbol de Widgets (sin azúcar sintáctica)
<code>npm install</code>	<code>flutter pub add</code>

La diferencia más importante: **Dart compila a AOT (ahead-of-time)** a binario nativo, no se interpreta en runtime. Por eso la app es rápida y los `print` en producción no aparecen a menos que uses `kDebugMode`.

Lo que vienes escuchando de JavaScript encaja casi 1:1 con Dart. Si entiendes TypeScript, entiendes Dart en una tarde. Lo único nuevo es el **árbol de widgets** (la «vista») y la **reactividad** (cómo se redibuja la pantalla cuando cambia un dato), que cubrimos más adelante.

1.3. Vistazo del repositorio

La raíz tiene tres directorios que importan para nosotros:

```
bitets/  
├── lib/           # todo el código de la app (Dart)  
│   ├── main.dart # punto de entrada  
│   └── core/      # infraestructura compartida
```

```

├── features/      # features (auth, examen, grid, ...)
├── assets/        # imágenes y assets
└── pubspec.yaml  # equivalente a package.json

```

pubspec.yaml es como package.json de npm o go.mod de Go: declara dependencias y configuración. Las dependencias se instalan con `flutter pub get` y los archivos autogenerados (los *.g.dart y *.freezed.dart) se regeneran con `dart run build_runner build --delete-conflicting-outputs`.

La parte interesante es lib/. Ahí vive todo. Dentro hay dos carpetas:

- **core/**: infraestructura compartida por todas las features (cliente HTTP, base de datos local, tema visual, router, notificaciones, constantes).
- **features/**: una carpeta por cada «feature» del producto. Cada feature tiene a su vez tres subcarpetas: **data/**, **domain/**, **presentation/**.

2. Arquitectura: Clean Architecture versión Flutter

Si vienes de Laravel, la analogía más cercana es **arquitectura hexagonal** (o «puertos y adaptadores»). Si vienes de Go, es la estructura habitual de un servicio con **internal/domain**, **internal/service** y **internal/repository**. Si vienes de NestJS, es la separación entre **controllers**, **services** y **entities** pero más estricta.

2.1. Las tres capas

Capa	Qué contiene	Analogía
domain/	Entidades de dominio + interfaces de repositorios	Las struct y las interface de Go. La verdad del negocio, sin dependencias externas.
data/	Implementaciones: datasources (HTTP/local) y repositorios	La capa de repository de Laravel o el internal/repository de Go: aquí viven el cliente HTTP, la base de datos local, y la implementación concreta de las interfaces de domain/.
presentation/	Widgets, páginas, providers	Los controllers y pages de Next.js. Maneja UI, eventos del usuario, y estado de pantalla.

La regla de oro, igual que en cualquier proyecto clean: **las dependencias apuntan hacia adentro**. presentation puede usar domain y data, data puede usar domain, pero domain no sabe nada de las otras dos.

2.2. Ejemplo: la feature auth

Para que la cosa no quede abstracta, miremos lib/features/auth/:

```

features/auth/
├── domain/
│   ├── entities/user.dart          # la entidad User (POJO con @JsonSerializable)
│   └── repositories/auth_repository.dart # interface del repo
├── data/
│   ├── datasources/
│   │   ├── auth_remote_datasource.dart # llama al backend con Dio
│   │   └── auth_local_datasource.dart  # lee/escribe secure storage
│   ├── models/
│   │   ├── auth_response_model.dart    # DTO: { user, token }
│   │   ├── user_model.dart             # espejo de la entity para JSON
│   │   ├── login_request.dart          # body del POST /auth/login
│   │   └── register_request.dart       # body del POST /auth/register
│   └── repositories/

```

```

└── auth_repository_impl.dart      # implementa AuthRepository
└── presentation/
    ├── pages/
    │   ├── login_page.dart        # pantalla de login
    │   ├── unlock_page.dart       # pantalla de desbloqueo biométrico
    │   ├── splash_page.dart       # pantalla inicial mientras carga
    │   ├── home_page.dart         # home del alumno
    │   └── profile_page.dart       # perfil
    ├── widgets/
    │   ├── login_form.dart
    │   ├── register_form.dart
    │   └── biometric_setup_dialog.dart
    └── providers/
        ├── auth_provider.dart     # @riverpod class Auth (state notifier)
        └── auth_state.dart        # @freezed sealed class AuthState

```

El patrón se repite en todas las features. `areas/`, `profesores/`, `carrera/`, `edificio/`, `examen/`, `plan_estudio/`, `salon/`, `unidad_aprendizaje/` siguen exactamente la misma forma.

2.3. Una sutileza: entidad vs modelo

Hay dos clases que representan «lo mismo»: `User` (en `domain/`) y `UserModel` (en `data/`). ¿Por qué? Es el mismo patrón de Laravel con Eloquent: el modelo de datos (cómo se ve el JSON) y la entidad de dominio (cómo lo usa la lógica de negocio).

```

// domain/entities/user.dart – la verdad del negocio
@JsonSerializable()
class User {
  final int id;
  final String name;
  final String email;
  final String? identificador;
  final String rol;
  // ...
}

// data/models/user_model.dart – DTO idéntico, vive en data
@JsonSerializable()
class UserModel {
  final int id;
  final String name;
  final String email;
  final String? identificador;
  final String rol;
  // ...
  domain.User toEntity() => domain.User(...);
}

```

`UserModel` sabe «soy un JSON», `User` sabe «soy un usuario del sistema». El repositorio convierte uno en otro. En la práctica aquí son casi iguales, pero la separación te permite cambiar la API sin tocar el dominio.

3. Stack de librerías: qué usamos y por qué

Esto es como el `package.json`: una lista de dependencias y para qué sirve cada una. Lo bueno es que casi todo el código de `lib/` termina siendo **orquestación** entre estas librerías. Si entiendes qué hace cada una, entiendes la app.

Librería	Versión	Qué hace en la app
flutter_riverpod	3.1.0	Manejo de estado global. Equivalente a Pinia/Vuex/Redux, pero sin Provider raíz obligatorio y con ref.watch que se siente como useSWR.
riverpod_annotation	4.0.0	Anotaciones para generar providers con código. Como los decorators de NestJS o los tags de Go.
riverpod_generator	4.0.0	Generador de código. Produce los *.g.dart a partir de @riverpod.
dio	5.4.0	Cliente HTTP. Como axios en JS o net/http en Go. Es el que habla con el backend.
go_router	14.8.0	Router declarativo. Como React Router o Vue Router: declaras rutas y el navegador (o la app) navega entre ellas.
freezed + freezed_annotation	3.x	Generador de data classes inmutables con copyWith, ==, map, when. Como los records de C# o los data class de Kotlin.
json_serializable + json_annotation	6.8.0	Generador de fromJson/toJson a partir de una clase. Como class-transformer de NestJS.
flutter_secure_storage	10.0.0	Almacenamiento cifrado (Keychain en iOS, Keystore en Android, libsecret en Linux). Donde guardamos el token JWT.
local_auth	2.3.0	Wrapper de biometría (huella, Face ID). Como navigator.credentials o el BiometricPrompt de Android.
device_info_plus	13.0.0	Info del dispositivo para enviar al backend como device_name.
drift + drift_flutter + drift_dev	2.34.0	ORM tipado sobre SQLite. Como Prisma pero con queries en Dart y migraciones automáticas.
connectivity_plus	7.1.1	Detecta cambios de red. Lo usamos para sincronizar borrados diferidos cuando vuelve el internet.
flutter_local_notifications + timezone + flutter_timezone	22 / 0.11 / 5.1	Notificaciones locales programadas (con hora exacta). Equivalente a Notification.schedule de Web pero nativo.
share_plus	13.1.0	Compartir archivos (usado para descargar el .ics de un examen).
open_filex	4.5.0	Abrir un archivo con la app por defecto del sistema.
path_provider	2.1.4	Resolver rutas de filesystem (carpeta temporal, documentos).
build_runner	2.4.13	Orquestador de generación de código. Como tsc --watch pero para paquetes Dart. Se invoca con dart run build_runner build.

Librería	Versión	Qué hace en la app
flutter_lints	6.0.0	Reglas de lint. Como eslint o golangci-lint.
custom_lint + riverpod_lint	—	Lints adicionales específicos de Riverpod. Te avisa si usas APIs deprecadas del framework.
flutter_launcher_icons	0.14.1	Genera los íconos de launcher para todas las plataformas a partir de assets/logo.webp.

4. La magia: generación de código en build time

Esta es la pieza que más confunde al que viene de JavaScript porque en JS/TS rara vez se genera código en build time. En bitets hay tres generadores corriendo al mismo tiempo. **No tienes que entender cada uno al detalle, pero sí saber que existen y cuándo se ejecutan.**

4.1. El comando sagrado

```
$ dart run build_runner build --delete-conflicting-outputs
```

Esto lee todos los archivos .dart que tengan anotaciones especiales (@riverpod, @JsonSerializable, @freezed, @DriftDatabase), los procesa, y produce archivos *.g.dart y *.freezed.dart junto al original. Esos archivos están **commiteados al repo** (es una decisión del proyecto) para que flutter analyze funcione sin tener que correr el generador primero.

4.2. @riverpod — providers tipados sin boilerplate

Un **provider** en Riverpod es como un useState global + un useEffect. Con la anotación @riverpod declaras una clase que se ocupa de crear y mantener ese estado. El generador produce el Provider que el resto de la app consume.

```
// providers/auth_provider.dart (lo que escribes tú)
@riverpod
class Auth extends _$Auth {
  @override
  AuthState build() {
    // setup, dependencias, estado inicial
    return const AuthState.initial();
  }

  Future<void> login(String id, String pass) async {
    // ...
  }
}

// providers/auth_provider.g.dart (lo que se genera)
final authProvider = AuthProvider._();
```

En la UI consumes así:

```
final auth = ref.watch(authProvider); // estado actual
ref.read(authProvider.notifier).login(...); // invocar métodos
```

La parte no obvia: la clase se llama Auth y el provider generado authProvider. El generador respeta la convención «minúscula + camel case» y le agrega el sufijo Provider. La regla de oro: **si ves un @riverpod, busca el Provider con la misma raíz.**

4.3. @freezed — data classes inmutables con superpoderes

AuthState no es una clase común: es una **sealed class** (unión discriminada, como las Result<T, E> de Rust o las Maybe<T> de Haskell). Tiene cinco variantes: initial, loading, authenticated, unauthenticated, storedSession.

```
// providers/auth_state.dart (lo que escribes tú)
@freezed
class AuthState with _$AuthState {
  const factory AuthState.initial() = _Initial;
  const factory AuthState.loading() = _Loading;
  const factory AuthState.authenticated(User user) = _Authenticated;
  const factory AuthState.unauthenticated([String? message]) = _Unauthenticated;
  const factory AuthState.storedSession({
    String? userName,
    String? message,
  }) = _StoredSession;
}
```

El generador produce:

- == y hashCode correctos para que dos AuthState.authenticated(...) con el mismo user sean iguales.
- copyWith para clonar cambiando un solo campo.
- map, mapOrNull, when, maybeWhen, whenOrNull: funciones estilo **pattern matching** para inspeccionar el estado.

Así se usa en la UI:

```
final isLoading = authState.maybeWhen(
  orElse: () => false,
  loading: () => true,
);
```

Esto es equivalente a:

```
// JavaScript imaginario
const isLoading = authState.tag === 'loading' ? true : false;
```

Por qué se usa. Sin @freezed tendrías que escribir 200 líneas de boilerplate (constructores, ==, hashCode, copyWith, los métodos when/map). Con @freezed son 8 líneas declarativas.

4.4. @JsonSerializable — fromJson y toJson automáticos

Todas las clases que vienen de la API o van a ella usan esta anotación. El generador produce un _\$UserFromJson(Map<String, dynamic>) y un _\$UserToJson() que serializan correctamente.

```
@JsonSerializable()
class UserModel {
  final int id;
```

```

    final String name;
    @JsonKey(name: 'created_at') // mapea snake_case → camelCase
    final DateTime createdAt;
  }

  // Uso:
  final u = UserModel.fromJson(apiResponse.data);
  final body = u.toJson();

```

Para que la serialización de fechas funcione, hay un helper `jsonDecode/jsonEncode` con su `fromJson/toJson` definido en `json_annotation`. Dart no sabe serializar `DateTime` por defecto, así que la convención es enviar/recibir ISO 8601 ("2025-06-13T15:30:00Z") y `DateTime.parse` lo entiende.

4.5. @DriftDatabase — la base de datos local

Drift genera el SQL a partir de definiciones de tablas en Dart:

```

// core/database/app_database.dart
class ExamenesCache extends Table {
  IntColumn get id => integer();
  TextColumn get payload => text();
  DateTimeColumn get horario => dateTime();
  BoolColumn get activo => boolean().withDefault(const Constant(true));
  // ...
}

@DriftDatabase(tables: [ExamenesCache, UserCache, NotificacionExamen])
class AppDatabase extends _$AppDatabase {
  // ...
}

```

El generador produce las clases `ExamenesCacheData`, `ExamenesCacheCompanion`, las queries tipadas, y el código de migración. Tú escribes la tabla como un objeto Dart; el SQL se genera. Versión actual: `schemaVersion = 2`, con migración automática.

4.6. Resumen de archivos .g.dart

Cuando veas un part `'foo.g.dart'`; en la cabecera de un archivo, significa «este archivo tiene un gemelo generado automáticamente». El que abriste es el que escribió un humano; el `.g.dart` no se toca a mano (de hecho, la primera línea suele decir «GENERATED CODE — DO NOT MODIFY BY HAND»).

Tres sufijos según el generador:

Sufijo	Generador	Significado
<code>.g.dart</code>	<code>riverpod_generator</code> o <code>json_serializable</code> o <code>drift_dev</code>	Provider/clase generada automáticamente.
<code>.freezed.dart</code>	<code>freezed</code>	Implementación de la sealed class.

5. HTTP: cómo se habla con el backend

5.1. El singleton DioClient

Hay un solo cliente HTTP en toda la app. Vive en `lib/core/network/dio_client.dart`:

```

class DioClient {
  DioClient._(); // constructor privado
  static final Dio instance = _createDio(); // singleton

  static Dio _createDio() {
    final dio = Dio(
      BaseOptions(
        baseUrl: ApiConstants.baseUrl,
        connectTimeout: ApiConstants.timeout,
        receiveTimeout: ApiConstants.timeout,
        headers: {
          'Accept': 'application/json',
          'Content-Type': 'application/json',
        },
      ),
    );
    if (kDebugMode) {
      dio.interceptors.add(LogInterceptor(
        requestBody: true, responseBody: true, error: true,
      ));
    }
    return dio;
  }
}

```

Por qué es un singleton: para que los interceptores (la pieza que mete el token JWT en cada request) se registren una sola vez. **Nunca instales Dio con Dio() en otros archivos:** usa `DioClient.instance`. Es la regla 1 de la app.

Configurar la URL del backend. En `lib/main.dart::_resolveEndpoint()` se elige la URL según la plataforma. Por defecto es `http://127.0.0.1:8000/api/v1`. Para apuntar a otro backend sin recompilar, se puede sobrescribir con la variable `API_BASE_URL`. Para producción actual está fijada a `https://saets.nullpointer.us.kg/api/v1` en `lib/core/constants/api_constants.dart`.

5.2. El AuthInterceptor — el middleware que mete el token

```

// core/network/auth_interceptor.dart
class AuthInterceptor extends Interceptor {
  AuthInterceptor({
    required this.onUnauthorized,
    FlutterSecureStorage? storage,
  }) : _storage = storage ?? const FlutterSecureStorage();

  final FlutterSecureStorage _storage;
  final VoidCallback onUnauthorized;
  String? _cachedToken;

  static const _publicEndpoints = ['/auth/login', '/auth/register'];
  static const _unauthorizedSkipPaths = [
    '/auth/login', '/auth/register', '/auth/logout', '/auth/me',
  ];

  @override
  void onRequest(RequestOptions options, RequestInterceptorHandler h) async {
    final isPublic = _publicEndpoints.any(

```

```

        (path) => options.path.contains(path),
    );
    if (!isPublic && _cachedToken != null) {
        options.headers['Authorization'] = 'Bearer \$_cachedToken';
    }
    h.next(options);
}

@override
void onError(DioException err, ErrorInterceptorHandler h) async {
    if (err.response?.statusCode == 401) {
        final isAuthEndpoint = _unauthorizedSkipPaths.any(
            (path) => err.requestOptions.path.contains(path),
        );
        if (!isAuthEndpoint) {
            _cachedToken = null;
            await _storage.deleteAll();
            onUnauthorized();
        }
    }
    h.next(err);
}
}

```

Es el equivalente a un middleware de Express: para cada request saliente verifica si necesita token y lo agrega; para cada response con error, si es 401 (token expirado), limpia el storage y avisa al auth provider para que mande al usuario a /login.

La parte sutil: el interceptor tiene su **propio** cache en memoria del token (`_cachedToken`). Si lo leyera del secure storage en cada request, la app se sentiría lenta. El cache se actualiza con `setCachedToken` (tras login) y `clearCachedToken` (en logout).

El interceptor se registra una sola vez, dentro de `Auth.build()`:

```

@riverpod
class Auth extends _$Auth {
    @override
    AuthState build() {
        // ...
        _interceptor = AuthInterceptor(onUnauthorized: _handleUnauthorized);
        DioClient.addAuthInterceptor(_interceptor);
        return const AuthState.initial();
    }
}

```

5.3. El backend esperado

La app asume que el backend es un Laravel (o un API REST similar) con `apiResource` (rutas `/recurso`, `/recurso/{id}`) y devuelve el sobre estándar de Laravel:

```

// GET /areas?page=2
{
    "data": [ { "id": 1, "nombre": "...", "clave": "...", ... }, ... ],
    "links": { "first": "...", "last": "...", "prev": null, "next": null },
    "meta": { "current_page": 2, "last_page": 5, "total": 47, ... }
}

```

```

}

// GET /areas/1
{ "data": { "id": 1, "nombre": "...", ... } }

// POST /areas → 201 { "data": { ...el nuevo... } }

// DELETE /areas/1 → 204 (sin cuerpo)

```

6. Estado global: Riverpod 3.x en 5 minutos

6.1. El modelo mental

Imagina un `useState` global cuyo valor puedes leer desde cualquier widget, y cuya actualización dispara el redibujo solo de los widgets que dependen de él. Eso es un **provider** de Riverpod. La diferencia con `React Context` es que no hay «Provider» raíz obligatorio en el árbol: lees con `ref.watch` directamente desde un objeto `WidgetRef` que te entrega Flutter cuando tu widget es un `ConsumerWidget` o un `ConsumerStatefulWidget`.

Tres tipos que verás todo el tiempo:

Tipo	Cuándo se usa
<code>Provider<T></code>	Un valor singleton (por ejemplo, una instancia de repositorio). Equivalente a un <code>provide()</code> en Angular o un <code>getInstance()</code> en Laravel.
<code>FutureProvider<T></code>	Un valor que se obtiene con un <code>Future</code> (típicamente una llamada HTTP). Como un <code>useSWR</code> sin revalidación automática.
<code>@riverpod class X</code>	Una clase con estado mutable + métodos. Equivalente a un reducer de Redux o un store de Pinia.

6.2. ¿Quién le pasa ref a quién?

Un `WidgetRef` aparece cuando un widget es **Consumer**. Los más comunes:

- `ConsumerWidget` — un widget sin estado, equivalente a un function component con hooks.
- `ConsumerStatefulWidget` — un widget con estado, equivalente a un class component. Su `State` es `ConsumerState<...>`.
- `ref.read(provider)` — obtén el valor **una vez** (no se re-renderiza si cambia).
- `ref.watch(provider)` — suscríbete a cambios (re-renderiza cuando cambia).
- `ref.listen(provider, (prev, next) => ...)` — reacciona a cambios sin re-renderizar.

La regla práctica: usa `ref.watch` dentro de `build()`, `ref.read` dentro de callbacks (`onPressed`, `onTap`), y `ref.listen` para side-effects como navegar a otra pantalla o mostrar un `SnackBar`.

6.3. Ejemplo: el grid de áreas

```

// providers/areas_providers.dart
final areaRepositoryProvider = Provider<AreasRepository>(
  (ref) => AreasRepository(),
);

@riverpod
class AreasGrid extends _$AreasGrid {
  @override
  Future<PaginatedResult<Area>> build(int page) {
    final filters = ref.watch(areasFiltersProvider);
    return GridNotifierOps.refreshPage(

```

```

        ref.read(areaRepositoryProvider),
        page,
        query: filters.isEmpty ? null : filters,
    );
}

@riverpod
class AreasFilters extends _$AreasFilters {
    @override
    Map<String, dynamic> build() => {};
    void apply(Map<String, dynamic> filters) => state = filters;
}

```

Observa:

- `areaRepositoryProvider` es un **singleton** (se crea una vez).
- `AreasGrid.build(int page)` es un **family**: el `page` es el argumento. Cada página es una suscripción independiente.
- `ref.watch(areaFiltersProvider)` declara dependencia: si los filtros cambian, el grid se reconstruye.
- `GridNotifierOps.refreshPage` es un helper estático (no un provider) que llama al repo.

Family en Riverpod 3.x. Antes el argumento se leía con `arg`. En 3.x se lee con un getter que se llama igual que el parámetro: `page` (porque `build` se llama `build(int page)`). El generador crea `late final _$args = ref.$arg as int;` y luego un `int get page => _$args;`.

7. La pieza estrella: el sistema de grids reutilizable

El 80% del código «visible» de la app son grids (catálogos con listar / crear / editar / eliminar). En vez de duplicar lógica, hay una infraestructura genérica en `lib/features/grid/` y cada catálogo concreto (áreas, profesores, carreras, ...) hereda de ella.

7.1. Lo que el grid te da gratis

Cuando extiendes `GridPage<T>`, la base se ocupa de:

Feature	Cómo se implementa
Lista paginada	Lee la página actual de un provider, renderiza cards. <code>_PaginationBar</code> controla <code>_currentPage</code> con chevron prev/next y «Página X de Y».
Layout de card	Cada item se renderiza en un <code>Card</code> con el cuerpo que proporciones + un <code>PopupMenuButton</code> (<code>more_vert</code>) con las acciones.
Pull-to-refresh	<code>RefreshIndicator</code> con <code>AlwaysScrollableScrollPhysics</code> . El callback invalida el provider y espera su future.
Modal de confirmación	Antes de cualquier acción con <code>requiresConfirmation == true</code> muestra un <code>AlertDialog</code> (Cancelar / Confirmar).
Botón crear	Si la lista de acciones incluye un <code>CreateAction<T></code> , aparece un <code>IconButton</code> (<code>icon: Icons.add</code>) en el <code>AppBar</code> .
Estados loading/error	<code>state.when(loading, error, data)</code> muestra spinner, mensaje rojo, o la lista (o «Sin resultados» si está vacía).
Refresh tras mutar	Tras una acción exitosa, llama a <code>widget.onActionCompleted(ref)</code> que típicamente invalida el provider.

Feature	Cómo se implementa
Submit del formulario + error	El botón Guardar del form hace POST o PUT con <code>DioClient.instance</code> . Errores 4xx/5xx se muestran en un <code>errorContainer</code> con el mensaje del backend.

7.2. Anatomía del grid: cinco archivos por catálogo

Para añadir un catálogo «Foo» backed por /foos necesitas cinco archivos (más uno de búsqueda opcional). Veamos el caso real de areas/:

Archivo	Qué contiene
features/areas/domain/entities/area.dart	La entidad Area <code>extends HasId</code> con su <code>fromJson</code> manual.
features/areas/data/repositories/areas_repository.dart	<code>class AreasRepository extends GridRepositoryImpl<Area></code> con <code>fromJson</code> sobrescrito.
features/areas/presentation/providers/areas_providers.dart	Un <code>Provider<AreasRepository></code> , un <code>@riverpod class AreasGrid</code> , y un <code>@riverpod class AreasFilters</code> .
features/areas/presentation/forms/area_form.dart	El formulario: tres campos, <code>hydrate</code> , <code>collectFormData</code> .
features/areas/presentation/pages/areas_grid_page.dart	La página concreta que extiende <code>GridPage<Area></code> y provee el título, el repositorio, las acciones, el <code>formBuilder</code> y el <code>buildCardBody</code> .

Mira el repositorio: ¿notas que son 11 líneas? La infraestructura genérica se ocupa de todo:

```
class AreasRepository extends GridRepositoryImpl<Area> {
  AreasRepository()
    : super(controller: const LaravelResourceController('/areas'));

  @override
  Area fromJson(Map<String, dynamic> json) => Area.fromJson(json);
}
```

7.3. LaravelResourceController — el builder de URLs

```
class LaravelResourceController {
  const LaravelResourceController(this.basePath);
  final String basePath;

  String list({int? page, Map<String, dynamic>? query}) {
    // ?page=2&otro=valor
  }
  String show(String id)    => '\$basePath/\$id';
  String create()           => basePath;
  String update(String id)  => '\$basePath/\$id';
  String delete(String id)  => '\$basePath/\$id';
  String attach(String id)  => '\$basePath/\$id';
}
```

Es el constructor de URLs a partir de un `basePath` ('/areas', '/profesores', '/mis-examenes', ...). Cada catálogo instancia uno con su path. La pieza es importante porque cuando añades un endpoint nuevo (digamos `restore` para `soft-deletes`) solo tocas este archivo y queda disponible para todos.

7.4. `GridAction<T>` — el menú de 3 puntos

El menú contextual de cada card no se construye con un `if/else`; cada acción es una clase. La base tiene cuatro:

```
abstract class GridAction<T extends HasId> {
  const GridAction();
  String get label;
  IconData get icon;
  bool get requiresConfirmation => false;
  String? get confirmationMessage => null;
  bool get requiresAdmin => false;      // ← bitets añadió esto
  bool isVisibleForItem(ref, item) => true;
  Future<bool> execute(context, item, repo, controller, formBuilder);
}
```

- `CreateAction<T>`: empuja un form vacío.
- `ViewAction<T>`: empuja el form en modo read-only.
- `EditAction<T>`: empuja el form con datos prellenados.
- `DeleteAction<T>`: borra y devuelve `true` para refrescar.

Pero se pueden añadir acciones custom. En `examen/` hay cuatro acciones custom (`AddReminderAction`, `AddToCalendarExamenAction`, `UnenrollExamenAction`, `ViewExamenDetailsAction`) que no son CRUD puro: navegar, mostrar un diálogo, compartir un archivo. La abstracción no te obliga a hacer CRUD; te obliga a tener el método `execute` que devuelve `Future<bool>` («¿debo refrescar al volver?»).

Detalle de `requiresAdmin`. El proyecto añadió este getter porque ciertos grids (por ejemplo «restaurar un elemento borrado») solo deben estar disponibles para `rol == 'admin'`. El `GridState` filtra las acciones con un `where((a) => !a.requiresAdmin || _isAdmin(ref))`. Es la única variante que el repo de bitets introduce sobre el patrón genérico.

7.5. `HasId` y `PaginatedResult<T>` — los contratos mínimos

```
abstract class HasId {
  const HasId();
  String get id;
}

class PaginatedResult<T extends HasId> {
  final List<T> items;
  final int currentPage;
  final int lastPage;
  final int total;
  bool get hasNextPage => currentPage < lastPage;
  bool get hasPreviousPage => currentPage > 1;
}
```

`HasId` es lo único que el grid te exige a tu modelo. Como el backend manda ids numéricos y Dart usa `String` por seguridad, el `fromJson` hace la conversión. `PaginatedResult<T>` se construye a partir del meta de Laravel.

7.6. El form genérico

GridForm<T> y GridFormState<T> son la pareja abstracta. Tú extiendes y solo implementas:

```
class AreaForm extends GridForm<Area> {
  const AreaForm({
    super.key,
    required super.endpoint,
    super.item,
    super.readOnly = false,
  });
  @override
  ConsumerState<GridForm<Area>> createState() => _AreaFormState();
}

class _AreaFormState extends GridFormState<Area> {
  final _nombreController = TextEditingController();
  final _claveController = TextEditingController();
  final _observacionesController = TextEditingController();

  @override
  String get formTitle =>
    widget.item == null ? 'Nueva area' : 'Editar area';

  @override
  void hydrate(Area? item) {
    _nombreController.text = item?.nombre ?? '';
    _claveController.text = item?.clave ?? '';
    _observacionesController.text = item?.observaciones ?? '';
  }

  @override
  Widget buildFormFields(BuildContext context, Area? item) {
    return Column(
      crossAxisAlignment: CrossAxisAlignment.stretch,
      children: [
        TextField(controller: _nombreController, ...),
        // ...
      ],
    );
  }

  @override
  Map<String, dynamic> collectFormData() {
    final nombre = _nombreController.text.trim();
    // ... en update, omite strings vacíos para que
    // la validación "sometimes" del backend no borre campos
    if (widget.item == null) {
      return { 'nombre': nombre, 'clave': clave, ... };
    }
    return {
      if (nombre.isNotEmpty) 'nombre': nombre,
      if (clave.isNotEmpty) 'clave': clave,
      // ...
    };
  }

  @override
```

```

void dispose() {
  _nombreController.dispose();
  _claveController.dispose();
  _observacionesController.dispose();
  super.dispose();
}
}

```

GridFormState se ocupa de:

- El AppBar con el botón «Guardar» (escondido si readOnly).
- AbsorbPointer(absorbing: readOnly) para que en modo «Ver» no se pueda editar.
- El POST o PUT con DioClient.instance (POST si item == null, PUT si no).
- Navigator.pop(context, true) al éxito.
- Mostrar el error con _extractDioMessage (helper que interpreta la respuesta de Laravel: busca data.message, fallback por código HTTP).

8. Almacenamiento local: Drift (SQLite tipado)

8.1. El schema

lib/core/database/app_database.dart define tres tablas:

```

class ExamenCache extends Table {
  IntColumn get id => integer();
  TextColumn get payload => text(); // JSON completo del Examen
  TextColumn get descripcion => text().withDefault(const Constant(''))();
  DateTimeColumn get horario => dateTime();
  BoolColumn get activo => boolean().withDefault(const Constant(true))();
  BoolColumn get pendingDelete => boolean().withDefault(const Constant(false))();
  DateTimeColumn get pendingDeleteAt => dateTime().nullable();
  DateTimeColumn get cachedAt => dateTime();
  @override
  Set<Column<Object>> get primaryKey => { id };
}

class UserCache extends Table {
  IntColumn get id => integer();
  TextColumn get payload => text();
  DateTimeColumn get cachedAt => dateTime();
  @override
  Set<Column<Object>> get primaryKey => { id };
}

class NotificacionExamen extends Table {
  IntColumn get id => integer().autoIncrement();
  IntColumn get examenId => integer();
  IntColumn get notificationId => integer();
  TextColumn get tipo => text();
  DateTimeColumn get fireAt => dateTime();
  BoolColumn get cancelled => boolean().withDefault(const Constant(false))();
  DateTimeColumn get createdAt => dateTime();
}

```

@DriftDatabase(tables: [...]) le dice al generador qué tablas incluir. Drift produce ExamenCacheData, ExamenCacheCompanion, y los DAOs tipados.

¿Por qué guardar el payload como texto? Cada examen se serializa a JSON y se guarda en una sola columna payload. Los demás campos (descripcion, horario, activo) son **índices** para poder filtrar/ordenar sin parsear JSON. Esto es el patrón «table-as-cache»: la tabla es solo un caché local; la fuente de verdad sigue siendo el backend.

8.2. Provider y ciclo de vida

```
// core/database/database_provider.dart
final appDatabaseProvider = Provider<AppDatabase>((ref) {
  final db = AppDatabase.defaults(); // abre SQLite en path nativo
  ref.onDispose(() async { await db.close(); });
  return db;
});
```

drift_flutter resuelve la ruta del archivo (en Android es `getApplicationSupportDirectory()` + nombre, en Linux es `~/.local/share/...`). El provider cierra la DB cuando el contenedor de Riverpod se descarta (típicamente al cerrar la app).

8.3. Los datasources locales

Hay dos fuentes de datos locales que envuelven la DB:

Datasource	Responsabilidad
UserLocalDatasource	Guarda y recupera un único usuario en la tabla userCache (id fijo = 1, upsert con <code>insertOnConflictUpdate</code>).
ExámenesLocalDatasource	CRUD sobre exámenes en <code>exámenes_cache</code> . Tiene <code>watchAll()</code> (Stream para refrescar UI reactiva), <code>replaceAll(List)</code> en transacción, <code>markPendingDelete</code> , <code>removeRow</code> , y <code>getPendingDeletes</code> para sincronizar borrados diferidos.

El truco del `pendingDelete`: cuando el usuario da «Desinscribirse» y no hay internet, no perdemos el examen de la UI (lo marcamos `pendingDelete = true` y queda oculto por el `where`). Cuando vuelve la conexión, `ExámenesSyncService` (que se suscribe a `Connectivity().onConnectivityChanged`) hace el `DELETE` al backend y remueve la fila local.

9. Notificaciones: recordatorios de exámenes

`lib/core/notifications/notifications_service.dart` es la pieza que programa notificaciones locales. Solo corre en móvil (Android, iOS, macOS); en otros sistemas devuelve `true` sin hacer nada.

9.1. Flujo general

```
// main.dart – al arrancar
notifications.initialize();
notifications.requestPermissions();
notifications.rescheduleAll(); // re-programa las que siguen pendientes
```

Cuando el alumno se inscribe a un examen, el repositorio `AlumnoExamenRepository` llama a `scheduleAt(examenId, fireAt, title, body, tipo)`. Esto:

1. Decide el `fireAt` (por defecto 3 horas antes, o 1 minuto antes si el examen está muy próximo).
2. Genera un `notificationId` único combinando `examenId` y el minuto de disparo.
3. Llama a `FlutterLocalNotificationsPlugin.zonedSchedule(...)` usando la zona horaria local (`tz.local`) resuelta con `flutter_timezone`.

4. Persiste la programación en la tabla `notificacionExamen` para poder `rescheduleAll` después de un reinicio.

Para cancelar, `cancelForExamen(examenId)` busca todas las filas de la tabla para ese examen, llama a `_plugin.cancel(id: ...)` y marca las filas como `cancelled = true`.

9.2. Zona horaria

La inicialización hace algo delicado:

```
tz_data.initializeTimeZones();
final tzInfo = await FlutterTimezone.getLocalTimezone();
tz.setLocalLocation(tz.getLocation(tzInfo.identifier));
```

Si esto falla (raro en móvil, común en escritorio), la app no se cae: usa UTC como fallback. Esto es importante porque `tz.TZDateTime.from(fireAt, tz.local)` necesita saber la zona para disparar la notificación a la hora local del usuario, no a UTC.

10. Router: navegación declarativa

`lib/core/router/app_router.dart` usa `go_router`, que es el equivalente Flutter de React Router o Vue Router. Hay cinco rutas:

```
routes: [
  GoRoute(path: '/', builder: (_, __) => const SplashPage()),
  GoRoute(path: '/login', builder: (_, __) => const LoginPage()),
  GoRoute(path: '/unlock', builder: (_, __) => const UnlockPage()),
  GoRoute(path: '/home', builder: (_, __) => const HomePage()),
  GoRoute(path: '/admin', builder: (_, __) => const AdminHomePage()),
]
```

Lo interesante es el **redirect**, que decide a dónde enviar al usuario según el estado de auth:

```
redirect: (context, state) {
  final auth = authListenable.value;
  final loc = state.matchedLocation;

  // En vuelo: no mover al usuario
  final isLoading = auth.maybeWhen(
    orElse: () => false,
    loading: () => true,
  );
  if (isLoading) return null;

  final target = auth.maybeWhen(
    authenticated: (user) => _homeFor(user), // admin → /admin, otro → /home
    storedSession: (_, __) => '/unlock',
    unauthenticated: (_) => '/login',
    orElse: () => '/', // initial → splash
  );

  return loc == target ? null : target;
}
```

El truco del authListenable. El router usa refreshListenable: authListenable (un ValueNotifier que refleja el estado de authProvider). Esto evita que se reconstruya el GoRouter cada vez que cambia la auth (lo cual sería carísimo). ref.listen actualiza el ValueNotifier; GoRouter se entera y ejecuta el redirect.

11. Recorrido guiado de los flujos clave

Aquí es donde todo se ata. Vamos a seguir el camino de cuatro operaciones comunes desde el dedo del usuario hasta el backend y de vuelta.

11.1. Flujo 1: login con boleta y contraseña

Usuario abre la app → ve la pantalla /login.

1. main() corre → UncontrolledProviderScope → BitetsApp
2. BitetsApp.addPostFrameCallback → authProvider.checkAuthStatus()
3. checkAuthStatus():
 - lee secure storage
 - si no hay token → state = AuthState.unauthenticated()
 - si hay token → state = AuthState.storedSession(userName)
4. GoRouter.redirect ve storedSession y manda a /unlock
5. En /unlock, el UnlockPage llama loginWithBiometrics() automáticamente

Como no hay biometría configurada, el usuario tap «Iniciar sesión con otra cuenta» → va a /login.

6. LoginPage: tabs con LoginForm / RegisterForm
7. Usuario llena boleta + contraseña, tap "Iniciar sesion"
8. LoginForm._submit() → authProvider.notifier.login(boleta, pass)
9. Auth.login():
 - state = AuthState.loading()
 - deviceName = _getDeviceName() (Android: "samsung-SM-G998B")
 - await _repository.login(...) // ve abajo
 - state = AuthState.authenticated(user)
 - unawaited(_triggerSync()) // sincroniza examenes
10. AuthRepositoryImpl.login():
 - AuthRemoteDatasource.login(LoginRequest)
 - Dio.post('/auth/login', data: ...)
 - response: AuthResponseModel.fromJson
 - local.saveToken(response.token) // secure storage
 - local.saveUserInfo(user) // secure storage
 - userCache.saveUser(user) // SQLite
 - return user
11. AuthInterceptor.setCachedToken(token) // cache en memoria
12. GoRouter.redirect ve Authenticated → _homeFor(user)
 - admin/administrativo → /admin
 - alumno → /home

Todo el flujo tarda 200–500ms con un backend local. El loading() muestra el spinner; el AuthInterceptor ya tiene el token para futuras requests.

11.2. Flujo 2: listar áreas (admin)

Admin abre el drawer, tap «Áreas».

1. AdminHomePage._selectPage(2) → setState(_currentIndex = 2)
2. body: _pages[2] → AreasGridPage()
3. AreasGridPage extiende GridPage<Area> con:
 - title = 'Areas'
 - repository = AreasRepository()
 - controller = LaravelResourceController('/areas')
 - actions = [Create, View, Edit, Delete]
 - watchGrid: ref.watch(areasGridProvider(page))
4. areasGridProvider.build(1) corre:
 - lee areasFiltersProvider (vacío al inicio)
 - GridNotifierOps.refreshPage(repo, 1, query: null)
 - AreasRepository.fetchPage(1)
 - GridRepositoryImpl.fetchPage
 - LaravelGridDatasource.fetchPage(1)
 - Dio.get('/areas?page=1') ← con Bearer token
 - return LaravelPaginatedResponse.fromJson(...)
 - map data: List<Area> via AreasRepository.fromJson
 - return PaginatedResult<Area>
5. AsyncValue<PaginatedResult<Area>>.data(...) se renderiza
6. ListView de Cards con PopupMenuButton
7. _PaginationBar abajo: "Pagina 1 de 3", "Total: 47"

11.3. Flujo 3: editar un área

Admin tap 3 puntos en la card «Sistemas» → «Editar».

1. PopupMenuButton.onSelected(EditAction) → _runAction(EditAction, item)
2. EditAction.requiresConfirmation == false, salta el dialog
3. EditAction.execute(context, item, repo, controller, formBuilder):
 - Navigator.push(
 - MaterialPageRoute(
 - builder: (_) => formBuilder(
 - endpoint: controller.update(item.id), // '/areas/1'
 - item: item,
 - readOnly: false,
4. AreaForm con endpoint='/areas/1', item=Area
5. AreaFormState.initState → hydrate(item) → controllers pre-llenos
6. Usuario edita el nombre, tap "Guardar"
7. GridFormState._submit():
 - data = collectFormData() // solo campos no vacíos
 - await _dio.put('/areas/1', data: data)
 - response 200 → Navigator.pop(context, true)
8. Back en GridState: action devuelve true → onActionCompleted(ref)
 - areasGridPage.onActionCompleted:
 - ref.invalidate(areasGridProvider)
 - ref.invalidate(areasListProvider) // por si otro form lo usa
9. AreasGrid.build(1) corre de nuevo → lista actualizada

11.4. Flujo 4: alumno abre la app sin internet

Este es el flujo más interesante. Asume que el alumno ya hizo login antes y luego pierde la conexión.

1. App abre, hay token en secure storage
2. `checkAuthStatus()` → `AuthState.storedSession(userName)`
3. `GoRouter.redirect` → `/unlock`
4. `UnlockPage.initState` → `_tryBiometric()`
5. `Auth.loginWithBiometrics()`:
 - `state = AuthState.loading()`
 - `LocalAuthentication.authenticate(...)` ← funciona offline
 - si ok: `interceptor.setCachedToken(token)`
 - `await _repository.getCurrentUser()`
 - `AuthRemoteDatasource.getCurrentUser()` → `Dio.get('/auth/me')`
 - *sin internet* → `DioException`
 - catch (`DioException 401`): logout, ir a `/login`
 - catch (otro): log debug, no cambia estado
 - revisa `_repository.getCachedUser()` ← SQLite `userCache`
 - si hay cache: `state = AuthState.authenticated(cached)`
 - si no: `state = storedSession(message: 'sin internet')`
6. Alumno entra a `/home` → `HomePage` → `AlumnoExamenesGridPage`
7. `AlumnoExamenesGridPage.watchGrid(1)` → `alumnoExamenesGridProvider.build(1)`
8. `AlumnoExamenRepository.fetchPage(1)`:
 - `_hasConnection()` ← `Connectivity().checkConnectivity()` → none
 - `isOnline = false`
 - NO sincroniza
 - `_paginateLocal(1)` ← lee de SQLite
 - devuelve `PaginatedResult` con los exámenes cacheados
9. Alumno ve su lista (sin updates del backend, pero funcional)
10. Cuando vuelve internet:
 - `ExamenesSyncService._onChange` detecta online
 - `syncPendingDeletes()`: borra del backend los marcados localmente con `pendingDelete = true`
 - Próxima vez que `AlumnoExamenRepository.fetchPage` corra, sincroniza los datos nuevos

Resumen de la estrategia offline. Los exámenes del alumno viven en SQLite. Cualquier operación CRUD que requiera backend marca la fila local como `pendingDelete` o la encola, y el `ExamenesSyncService` la reconcilia cuando vuelve la conexión. El alumno nunca queda sin información funcional.

12. Estructura detallada de carpetas

Lo que sigue es el árbol completo de `lib/`, con una nota para cada archivo o grupo. Si tienes que buscar algo, empieza por aquí.

```
lib/
├── main.dart                # punto de entrada, init de providers
├── core/                   # infraestructura compartida
│   ├── auth/
│   │   └── user_role.dart  # extension isAdmin/isStudent
│   ├── constants/
│   │   └── api_constants.dart # baseUrl, endpoints, timeouts
│   ├── database/
│   │   ├── app_database.dart # schema Drift (3 tablas)
│   │   └── app_database.g.dart # generado
```

```

├── database_provider.dart      # Provider<AppDatabase>
├── examenes_local_datasource.dart
├── user_local_datasource.dart
├── local_providers.dart       # datasources + sync service
├── network/
│   ├── dio_client.dart        # singleton Dio
│   └── auth_interceptor.dart   # mete Bearer + maneja 401
├── notifications/
│   └── notifications_service.dart # recordatorios locales
├── router/
│   └── app_router.dart         # GoRouter + redirect por auth
├── theme/
│   └── app_theme.dart          # Material 3 + seed #D96704
└── features/                  # una carpeta por feature
    ├── auth/                  # login, register, biometric, ...
    ├── grid/                  # infraestructura genérica (¡úsala!)
    ├── examen/                # el feature más complejo
    ├── areas/, profesores/, carrera/, edificio/, plan_estudio/,
    └── salon/, unidad_aprendizaje/, admin/ # los catálogos CRUD

```

Cada feature (excepto grid/ que es infraestructura) sigue el mismo patrón:

```

features/<nombre>/
├── data/
│   ├── datasources/          # llamadas HTTP / local storage
│   ├── models/               # DTOs @JsonSerializable (request/response)
│   └── repositories/         # implementación de la interface de domain
├── domain/
│   ├── entities/             # modelos de negocio (HasId, @JsonSerializable)
│   └── repositories/         # interfaces (abstract class)
└── presentation/
    ├── pages/                # pantallas
    ├── widgets/              # piezas reusables de UI
    ├── forms/                # formularios (extienden GridForm)
    ├── actions/              # GridAction<T> custom (opcional)
    ├── providers/            # @riverpod
    ├── services/             # lógica auxiliar (ej. calendar_export)
    └── providers.g.dart       # generado

```

13. Convenciones del proyecto (leer antes de tocar nada)

Regla	Detalle
Idioma	Español para strings visibles y debugPrint. Inglés para código, identificadores y comentarios técnicos.
Sin comentarios en código	Regla global: «DO NOT ADD ANY COMMENTS». El código debe entenderse solo. Si necesitas contexto, va al README o a este reporte.
Color primario	\#D96704 (naranja). Definido como seed en AppTheme.lightTheme. Para rebrand, toca solo ese archivo.
Material 3 siempre	useMaterial3: true en ThemeData. No uses APIs M2-only (FloatingActionButtonThemeData.sizeConstraints, etc).

Regla	Detalle
Endpoint paths centralizados	No hardcodees URLs en datasources. Usa <code>LaravelResourceController(basePath)</code> o las constantes en <code>api_constants.dart</code> .
Código generado se commitea	*.g.dart y *.freezed.dart van al repo. Reviewers rechazan diffs que no los incluyen.
Generar antes de commitear	Tras tocar archivos anotados, corre <code>dart run build_runner build --delete-conflicting-outputs</code> .
Riverpod 3.x	Usa <code>@riverpod</code> (no <code>StateProvider</code> manual). El argumento de <code>family</code> se llama como el parámetro de <code>build</code> (típicamente <code>page</code>), no <code>arg</code> .
No duplicar interceptors	Solo <code>Auth.build()</code> registra el <code>AuthInterceptor</code> . Otros providers no deben tocar <code>DioClient.instance.interceptors</code> .
Línea de comandos	<code>dart run build_runner build --delete-conflicting-outputs</code> <code>flutter analyze</code> <code>dart format lib/</code> <code>flutter test</code>

14. Glosario rápido (Dart/Flutter → JS/PHP/Go)

Término	Qué es	Tu analogía
Widget	Pieza fundamental de UI. Todo es un widget: botones, textos, padding, gestos. Los widgets se anidan como un árbol.	Componente de React. Pero más estricto: no hay <code>div</code> suelto, todo es una clase.
StatelessWidget	Widget sin estado mutable.	Function component de React.
StatefulWidget + State	Widget con estado mutable (un <code>setState</code> redibuja).	Class component de React con <code>this.state</code> .
BuildContext	Referencia a la posición de un widget en el árbol. Se pasa a métodos como <code>Navigator.of(context)</code> .	El <code>node</code> de React. Algunos helpers lo requieren para saber «dónde estoy».
Future	Representa un valor que llegará en el futuro (análogo a <code>Promise<T></code>).	<code>Promise<T></code> .
Stream	Secuencia de valores asíncronos. Como un <code>Observable<T></code> de RxJS.	<code>WebSocket</code> / <code>EventSource</code> / <code>Observable</code> .
Provider	Valor global con suscripción. En Riverpod 3.x, los creas con <code>@riverpod</code> o con <code>Provider<T>(...)</code> .	<code>Context</code> de React, <code>useStore</code> de Zustand, o <code>provide()</code> de Angular.
ConsumerWidget	Widget sin estado que puede leer providers.	Function component con <code>useContext</code> / <code>useStore</code> .
ConsumerStatefulWidget	Widget con estado que puede leer providers.	Class component que usa <code>useContext</code> (en realidad <code>Consumer</code>).
ref.watch / ref.read / ref.listen	Tres formas de consumir un provider. <code>watch</code> re-renderiza, <code>read</code> lee una vez, <code>listen</code> reacciona sin re-renderizar.	<code>useContext</code> / <code>useStore.getState</code> / <code>useEffect</code> .
Navigator.push / context.go	Navegación. <code>Navigator.push</code> para push apilables, <code>go_router</code> para rutas declarativas.	<code>history.push</code> / <code>React Router navigate</code> .

Término	Qué es	Tu analogía
Dart extension	Añade métodos a una clase que no controlas.	Los prototype extensions de JS, o los métodos de extensión de C#.
mixin	Código reutilizable que se «mezcla» en una clase.	Object.assign con prototipos, o traits de PHP.
sealed class	Jerarquía cerrada de subclases. El compilador puede hacer pattern matching exhaustivo.	Tagged unions de TypeScript, sealed de PHP 8.
build_runner	Generador de código en build time.	prisma generate, buf generate, protoc.
part "foo.g.dart";	Indica que el archivo actual se complementa con foo.g.dart generado.	No hay análogo directo; es como un \#include pero para código generado.

15. Resumen ejecutivo (para responder preguntas rápidas)

Si alguien te pregunta «¿cómo está hecha bitets?» y solo tienes 60 segundos:

- Es una app Flutter con Clean Architecture por features. Cada feature tiene data/, domain/, presentation/.
- Estado global con Riverpod 3.x usando @riverpod y generación de código (*.g.dart).
- HTTP con dio como singleton; un solo AuthInterceptor mete el JWT en cada request y maneja 401s.
- Backend asumido: Laravel con apiResource y el sobre estándar { data, links, meta }.
- 80% de las pantallas son grids. Hay una infraestructura genérica en lib/features/grid/ que todas las features extienden con solo 5 archivos por catálogo.
- Para datos persistentes: SQLite con Drift (esquema tipado + queries generadas). El caché de exámenes del alumno vive ahí.
- Notificaciones: flutter_local_notifications con zona horaria local; se persisten en una tabla para sobrevivir reinicios.
- Funciona offline: si no hay internet, los datos cacheados siguen disponibles, las eliminaciones se encolan y se sincronizan con connectivity_plus cuando vuelve la red.
- La única pantalla con un diseño realmente a medida es login/unlock/perfil; todo lo demás hereda de la base.

Fin del reporte. Si encuentras algo que no encaje con el código actual, el generador canónico de la verdad es el árbol en lib/ — un grep rápido sobre un símbolo siempre te lleva a la respuesta.